Tecnológico Nacional de México
Campus Querétaro

E10 - Diseñador de software

Producto de la asignatura:
Ingeniería de Software

Desarrolla por:

Díaz García Yaneli Guadalupe
22140752

Asesor docente:

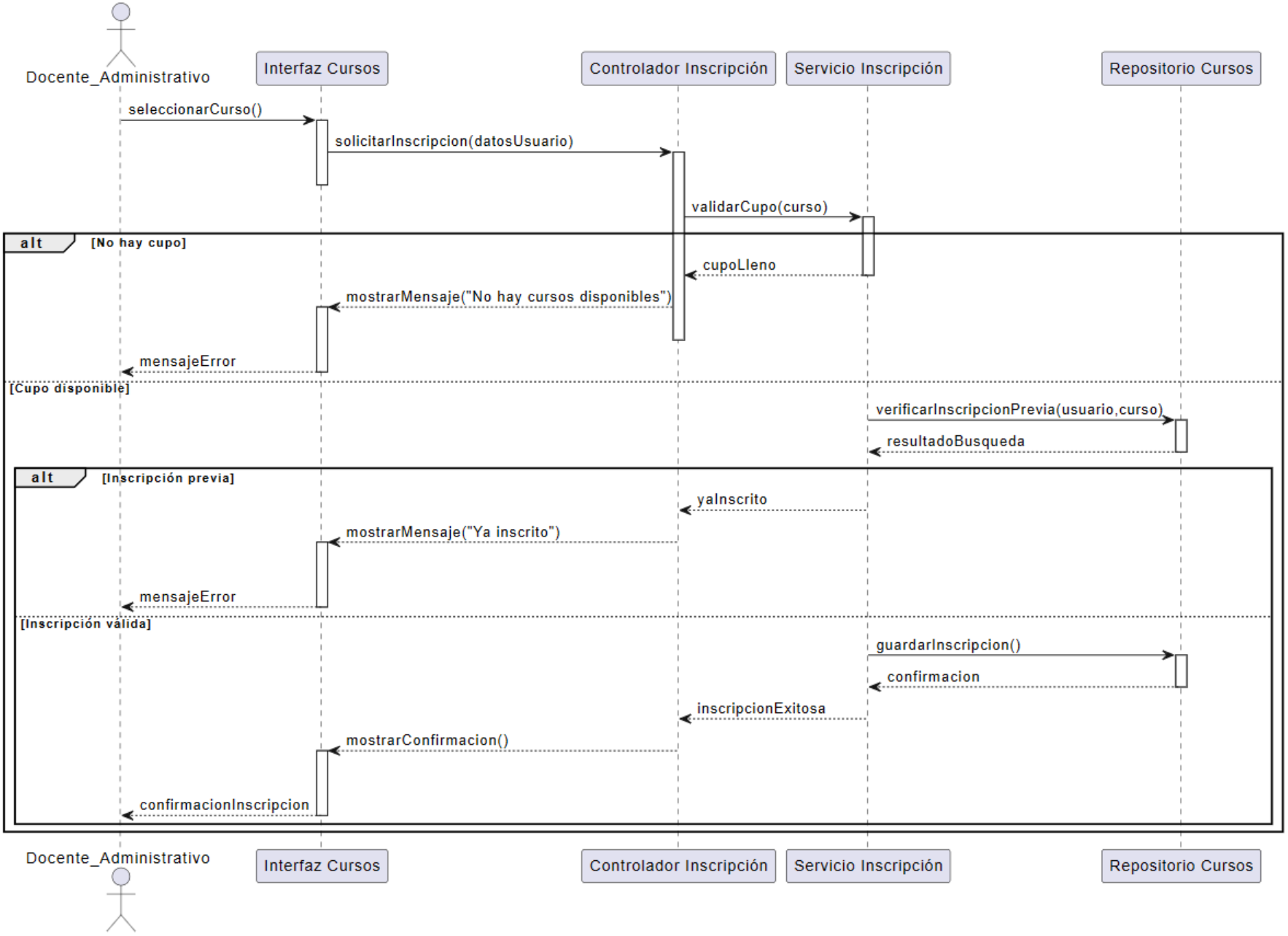
Villeda Maldonado Julio Alejandro

Fecha de entrega:

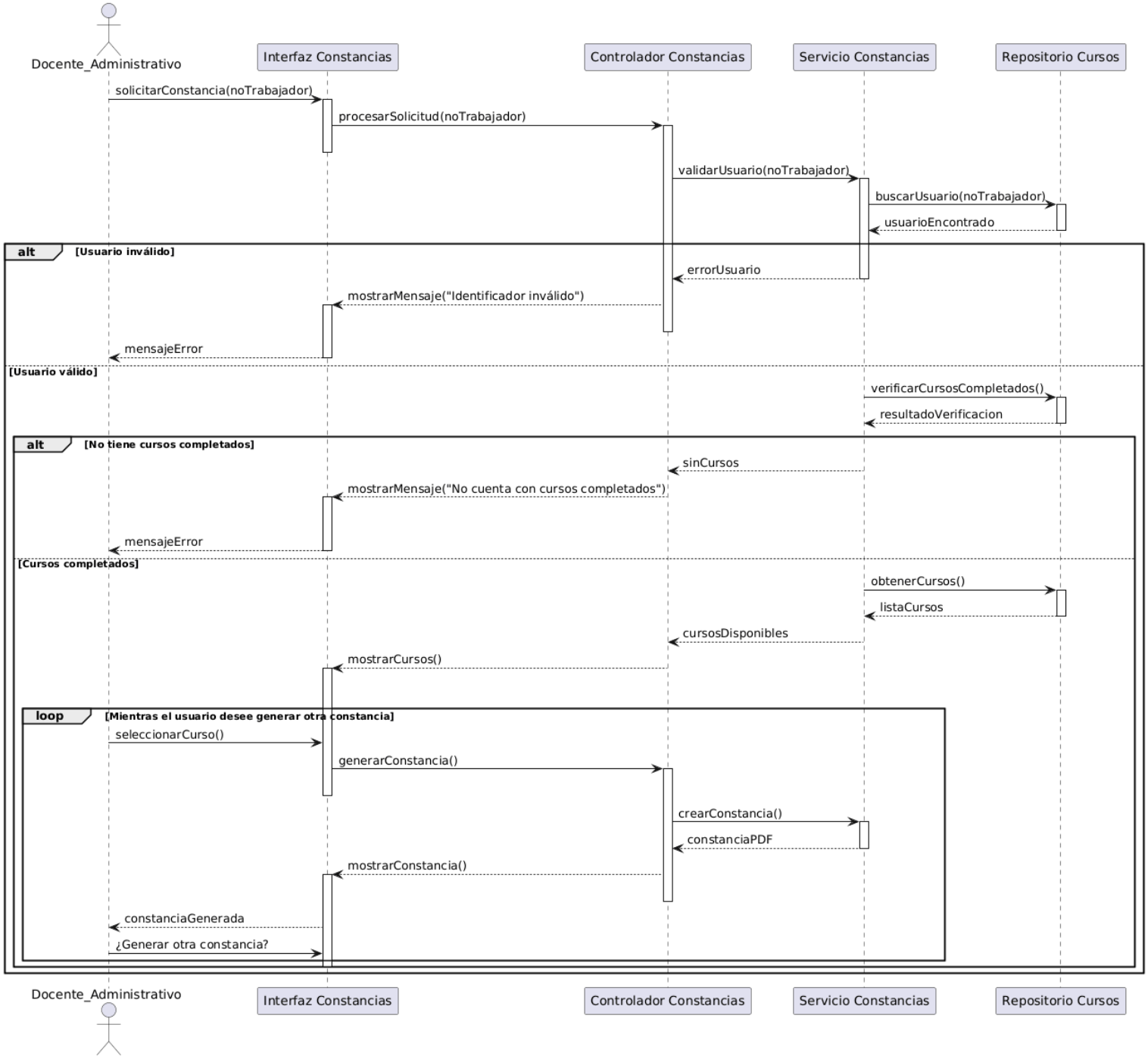
24 de febrero de 2026

Grupo:
6A

Caso Crítico: Inscribirse a un curso



Caso: Generar constancia



Justificación:

Se adoptó la arquitectura en capas Actor → Interfaz → Controlador → Servicio → Repositorio con el objetivo de separar responsabilidades y cumplir principios de diseño de software.

Caso de Uso: Inscribirse a curso

El diagrama inicia con el actor Docente/Administrativo, quien solicita la inscripción mediante la Interfaz Cursos, responsable únicamente de la interacción visual y captura de solicitudes. Posteriormente, la solicitud es delegada al Controlador de Inscripción, el cual coordina el flujo del caso de uso sin implementar reglas de negocio.

Las validaciones principales se ejecutan dentro del Servicio de Inscripción, cumpliendo con el principio arquitectónico de concentrar la lógica del negocio en la capa de servicios. Este componente verifica primero la disponibilidad del cupo del curso y posteriormente consulta al Repositorio Cursos para validar si existe una inscripción previa.

Se emplearon fragmentos combinados alt para representar correctamente los flujos alternos derivados de reglas funcionales previamente definidas:

- Curso sin cupo disponible.
- Usuario previamente inscrito.
- Inscripción válida.

Estos fragmentos permiten modelar decisiones reales del sistema equivalentes a estructuras IF–ELSE, manteniendo claridad en la secuencia temporal.

Caso de Uso: Generar constancia

El segundo diagrama modela la generación de constancias académicas, considerado un proceso crítico debido a la cantidad de validaciones involucradas.

El actor inicia la interacción solicitando la constancia mediante la Interfaz Constancias, la cual transfiere la solicitud al Controlador Constancias para su coordinación. El controlador delega la lógica al Servicio Constancias, encargado de validar la identidad del usuario y verificar el cumplimiento académico requerido.

El Repositorio Cursos actúa exclusivamente como capa de persistencia, consultando información almacenada sin contener lógica de negocio, respetando así las buenas prácticas arquitectónicas UML.

Se emplearon múltiples fragmentos alt para modelar condiciones relevantes del sistema:

- Usuario inválido.
- Usuario sin cursos completados.
- Usuario con cursos finalizados.

Caso de Uso: Inscribirse a curso

```
@startuml
actor Docente_Administrativo

participant "Interfaz Cursos" as UI
participant "Controlador Inscripción" as CI
participant "Servicio Inscripción" as SI
participant "Repositorio Cursos" as RC

Docente_Administrativo -> UI : seleccionarCurso()
activate UI

UI -> CI : solicitarInscripcion(datosUsuario)
activate CI
deactivate UI

CI -> SI : validarCupo(curso)
activate SI

alt No hay cupo
    SI --> CI : cupoLleno
    deactivate SI

    CI --> UI : mostrarMensaje("No hay cursos disponibles")
    activate UI
    deactivate CI

    UI --> Docente_Administrativo : mensajeError
    deactivate UI

else Cupo disponible
    SI -> RC : verificarInscripcionPrevia(usuario,curso)
    activate RC

    RC --> SI : resultadoBusqueda
    deactivate RC

    alt Inscripción previa
        SI --> CI : yaInscrito
        deactivate SI

        CI --> UI : mostrarMensaje("Ya inscrito")
        activate UI
        deactivate CI

        UI --> Docente_Administrativo : mensajeError
        deactivate UI

    else Inscripción válida
        SI -> RC : guardarInscripcion()
        activate RC

        RC --> SI : confirmacion
        deactivate RC

        SI --> CI : inscripcionExitosa
        deactivate SI

        CI --> UI : mostrarConfirmacion()
        activate UI
        deactivate CI

        UI --> Docente_Administrativo : confirmacionInscripcion
```

```
        deactivate UI
    end
end

@enduml
```

Caso de uso: Generar constancia

```
@startuml
actor Docente_Administrativo

participant "Interfaz Constancias" as UI
participant "Controlador Constancias" as CC
participant "Servicio Constancias" as SC
participant "Repositorio Cursos" as RC

Docente_Administrativo -> UI : solicitarConstancia(noTrabajador)
activate UI

UI -> CC : procesarSolicitud(noTrabajador)
activate CC
deactivate UI

CC -> SC : validarUsuario(noTrabajador)
activate SC

SC -> RC : buscarUsuario(noTrabajador)
activate RC
RC --> SC : usuarioEncontrado
deactivate RC

alt Usuario inválido
    SC --> CC : errorUsuario
    deactivate SC

    CC --> UI : mostrarMensaje("Identificador inválido")
    activate UI
    deactivate CC

    UI --> Docente_Administrativo : mensajeError
    deactivate UI

else Usuario válido
    SC -> RC : verificarCursosCompletados()
    activate RC
    RC --> SC : resultadoVerificacion
    deactivate RC

    alt No tiene cursos completados
        SC --> CC : sinCursos
        deactivate SC

        CC --> UI : mostrarMensaje("No cuenta con cursos completados")
        activate UI
        deactivate CC

        UI --> Docente_Administrativo : mensajeError
        deactivate UI

    else Cursos completados
        SC -> RC : obtenerCursos()
        activate RC
        RC --> SC : listaCursos
```

```
        deactivate RC

        SC --> CC : cursosDisponibles
        deactivate SC

        CC --> UI : mostrarCursos()
        activate UI
        deactivate CC

        loop Mientras el usuario desee generar otra constancia

            Docente_Administrativo -> UI : seleccionarCurso()

            UI -> CC : generarConstancia()
            activate CC
            deactivate UI

            CC -> SC : crearConstancia()
            activate SC

            SC --> CC : constanciaPDF
            deactivate SC

            CC --> UI : mostrarConstancia()
            activate UI
            deactivate CC

            UI --> Docente_Administrativo : constanciaGenerada

            Docente_Administrativo -> UI : ¿Generar otra constancia?
        end

        deactivate UI
    end
end

@enduml
```